

CPSC 471 Final Report: RentCA

Maryna Hez, Alexander Firth, Sage Kennard

1. Abstract

RentCA is a web-based rental property management system designed to support interactions between property managers, tenants, and prospective renters. The system provides a unified platform where users can browse listings, book viewing appointments, manage properties, track leases, process rent payments, and submit maintenance requests. The application is implemented using a React frontend, a Django REST backend, and a MySQL relational database. The system is structured around role-based functionality, where each user may act as a prospective renter, tenant, or property manager. Rather than dynamically changing roles, each role is stored as a separate user record, and users can switch between roles within the interface. This design simplifies role-specific operations while maintaining clear separation of responsibilities. Core functionalities include searching and filtering rental listings, booking appointments with date and time selection, and allowing property managers to confirm, cancel, or complete these appointments. Managers can also create lease records that link tenants to properties, enabling tenant-side features such as viewing lease details, making rent payments, and submitting maintenance requests. Several validation and consistency improvements were implemented to enhance system reliability. These include enforcing input constraints on user registration (such as name and password validation), preventing duplicate accounts with the same role and email, avoiding duplicate property entries, and ensuring that appointments cannot be double-booked for the same property and time. Additional checks were also added to prevent assigning multiple leases to the same property simultaneously. Overall, the system demonstrates a complete full-stack application that integrates a user-friendly interface with a relational database design, supporting realistic workflows in a rental management scenario.

2. Introduction

This project focuses on the design and implementation of a rental property management system intended to address common inefficiencies in managing rental operations. In high-demand rental markets such as Calgary, property managers and renters frequently rely on manual methods such as emails, phone calls, and spreadsheets to coordinate viewing appointments, track lease agreements, and manage tenant-related information. These approaches often result in scheduling conflicts, lost information, and a lack of reliable records.

As identified in our initial proposal, one of the most significant issues is the organization of viewing appointments. Multiple prospective renters may be scheduled for the same time slot, leading to confusion and unfair outcomes where units are effectively offered on a first-come

basis rather than through a structured process. In addition to appointment scheduling, rental management tasks such as lease tracking, rent payments, and maintenance requests are often handled using separate and inconsistent systems. This fragmentation makes it difficult for property managers to maintain accurate records and for tenants to track the status of their interactions. Updates are made instantaneously, so tenants and property managers avoid wait times and uncertainty when sending requests and getting direct responses.

To address these challenges, this project develops a centralized, database-driven web application that integrates all major aspects of rental property management into a single system. This system is designed to directly support multiple user roles: property managers, tenants, and prospective renters, and provides structured functionality for managing listings, booking appointments, tracking leases, recording payments, and handling management requests. By organizing all data within a relational database and enforcing constraints such as preventing double-booking of time slots, the system improves consistency, reduces errors, and provides a more reliable and efficient workflow for all users involved in the rental process.

3. System Description

To address the identified inefficiencies in rental property management, we developed a full-stack web application supported by a relational MySQL database. The system integrates property management, appointment scheduling, lease tracking, payment recording, and management requests into a single centralized platform.

The application follows a client-server architecture. The frontend is built using React with Vite, providing an interaction user interface for different user roles. The backend is implemented using Django and Django REST Framework, which exposes RESTful API endpoints for handling data operations and enforcing business logic. The system communicates with a MySQL database that stores all persistent data, including users, properties, listings, appointments, leases, payments, and requests.

A key feature of the system is the centralized handling of appointments. When a prospective renter books a viewing, the appointment is stored in the database and linked to both the user and the listing. Property managers can then update the appointment status (e.g., pending, confirmed, cancelled, completed), which is reflected in the renter's dashboard. The system design also incorporates constraints such as preventing inconsistent records and maintaining relationships between entities through foreign keys.

The database schema was implemented using structured SQL DDL statements, defining entities such as `USER`, `PROPERTY`, `RENTAL_LISTING`, `APPOINTMENT`, `TIME_SLOT`, `LEASE_RECORD`, `RENT_PAYMENT`, and `MANAGEMENT_REQUEST`. These tables are connected through primary and foreign key relationships to ensure data integrity and consistency across the entire system. Overall, the system provides a unified and structured

solution to rental management by replacing fragmented and manual processes with a centralized, database-driven application.

4. Project Design

The design of the RentCA system is centered around three main types of end-users: **Property Managers**, **Tenants** and **Prospective Renters**. These users interact with the same database-backed system, but each role is given a different interface, and a different set of available actions based on its responsibilities in the rental process. This role-based structure was chosen so that the system reflects the real-world interactions that occur before, during, and after tenancy.

A major design goal was to ensure that each type of user sees only the information and actions that are relevant to them. Instead of presenting one large interface with unnecessary options, the system adjusts the dashboard and navigation menu on the website according to the active role. This improves usability and makes the application easier to understand, especially because the project includes multiple related processes such as listing management, appointment booking, lease tracking, payment handling, and management requests

4.1 Property Manager

The Property Manager is the most operationally active user in the system. This role is responsible for maintaining rental information and responding to actions initiated by tenants and prospective renters. In practice, the Property Manager acts as the administrative side of the platform.

The manager's interface includes access to a dashboard, properties, listings, appointments, leases, payments, and requests. Through the **Properties** section, the manager can add a property, view existing properties, and filter them by city or rental status. The property data includes address-related fields such as province, street, name, postal code, suite, and apartment, which allows the system to distinguish between similar units in the same area. The manager can also view all properties currently under their responsibility. The **Listings** section allows the manager to create and manage rental advertisements tied to those properties just described. Unlike the prospective renter view, the manager can see not only active listings, but also rented and inactive ones. This allows the manager to track the full lifecycle of a listing rather than only the listings currently visible to the public side of the system. From a design point-of-view, this separation is important because a property and a listing are not treated as the same object in the system: a property is the physical rental unit, while a listing represents its advertised availability.

The **Appointments** section supports one of the main problems identified in the original proposal: unorganized viewing schedules. Property Managers can review appointments booked on their own listings, rather than seeing all appointments in the system. They can then update

appointment statuses using actions such as confirm, cancel, or complete. This design ensures that appointment control remains with the manager responsible for the relevant property, while prospective renters can still track outcomes through their own account. The appointment feature is therefore both a scheduling tool and a record-keeping mechanism.

The **Leases** section allows Property Managers to create lease records that associate a tenant with a property, including start date, end date, monthly rent, and security deposit. This is one of the most important transitions in the system, because it connects the property-management side of the application to the tenant side. Once a lease exists, the tenant can view lease-related details and payments in their own interface. The manager can see all current lease records in the same section, which gives them an overview of occupied properties and active rental agreements.

The **Payments** section gives managers oversight over rent payment records. Managers can filter payments by status, such as pending, completed, or failed, and can approve or fail submitted payments when needed. This design reflects the fact that payment tracking in rental systems is not only about recording that a payment exists, but also about recording its current state. The same design principle appears again in the **Requests** section, where managers can review management requests submitted by tenants, read their descriptions, and update their status to values such as pending, approved, rejected, or completed. Taken together, these features make the Property Manager role responsible for the active management of rental operations across the system.

4.2 Tenant

The Tenant role is designed around the responsibilities and information needs of someone who already occupies a property. While the Property Manager role is operational and administrative, the Tenant role is more focused on tracking existing rental obligations and communicating with management.

The tenant interface includes a dashboard with **My Leases**, **Payments**, and **My Requests**. The **My Lease** section gives the tenant access to the lease record created by the manager, including the associated property and lease details. This allows the tenant to view information such as the address, rental amount, security deposit, and lease dates in one place. A later refinement to the system also introduced support for identifying an *upcoming lease*, which corrected an earlier issue where lease display logic only reflected expired or active statuses. This improvement made lease status more information and aligned the interface more closely with real rental timelines.

The **Payments** section allows tenants to review rent payments associated with their lease and to submit new payment records. When creating a payment, the tenant selects the lease, enters an amount, chooses a payment method, and submits the payment. The date is filled automatically but can still be adjusted if needed. This design gives the tenant enough control to submit accurate payment information while still leaving review and status updates to the manager. In this sense, the tenant side of the payment system is intentionally asymmetric:

the tenant initiates the payment, but the Property Manager validates and manages its status afterwards to ensure a correct payment was made.

The **My Requests** section allows tenants to submit management or maintenance-related issues tied to a property. A tenant can create a new request by selecting the relevant property and writing a short description of the problem. Once submitted, the request becomes visible on both the tenant and manager sides of the application. This supports two important design goals at once: first, the tenant has a written record of the issue they submitted; second, the manager has a structured list of requests that can be tracked and updated over time. Because the system stores a status for each request, the tenant is able to return later and see whether the request is still pending, has been approved, has been rejected, or has already been completed.

A notable design limitation in the current system is that a Prospective Renter is not automatically converted into a Tenant when a lease is created. Instead, a separate Tenant account must already exist. In practice, this means that the same person may need both a Prospective Renter account and a Tenant account if they want to move through both stages of the process. This differs from the earlier project assumption that a Prospective Renter would become a Tenant upon signing a lease, but the final implementation keeps the roles as separate user records to simplify role-specific logic and interface behavior. The webface is designed to be easy for the user to switch between both these roles in the **Active Role** section of their dashboard, simply by clicking on the role they wish to become their primary role. Their dashboard will be updated to reflect whichever role is clicked as the active one.

4.3 Prospective Renter

The Prospective Renter role represents users who have not yet signed a lease and are still searching for available housing. This role focuses on discovery, browsing, and appointment booking rather than long-term property management.

The Prospective Renter interface includes a dashboard, **Browse Listings**, and **My Appointments**. In the **Browse Listings** section, users can search and filter rental listings using criteria such as city and price range. Only active listings are shown in this view, which keeps the renter-facing side of the application focused on currently available units rather than exposing inactive or already rented listings. This separation is important for both a usability and design perspective, because the information needs of prospective renters differs from those of managers or tenants. A renter wants to quickly find suitable available units that match their preferences, while a manager needs the broader history and management state of all their listings.

From the renter side, a listing is primarily an entry point into the appointment-booking workflow. The user can view listing information, choose to book a viewing, and select a preferred visit date. In the later version of the system, appointment booking also includes an hour selection, which was added to support the project's goal of preventing double-booking of

the same property at the same time. This was an important improvement since the earlier appointment design stored data information only at the day level. With the addition of time, the booking process becomes more realistic and better aligned with real-world scheduling problems that motivated the project.

The **My Appointments** section allows prospective renters to review the viewings they have scheduled and to monitor their status. This design gives renters a clear record of their interactions with managers and removes the uncertainty that can occur when appointments are handled informally through phone calls or messages. Instead of relying on outside communication, the system records each appointment directly and allows the renter to see whether it is still pending, has been confirmed, was cancelled, or has already been completed. This directly addresses one of the central problems described in the original proposal: overlapping appointments, inconsistent records, and lack of reliable scheduling information.

4.4 Role-Based Design Considerations

Although the three user roles are different, they are closely connected. A Property Manager creates the properties and listings that a Prospective Renter sees. A Prospective Renter can then book an appointment on one of those listings. If the rental process continues, a lease can be created for a Tenant, and that lease then supports future payment and request activity. This means the user design was not treated as three isolated interfaces, but rather as three connected parts of the same rental workflow.

Another important design choice is that the system allows an individual to hold multiple roles through separate user records. Once logged in, the user can switch between roles that belong to the same identity (account) and see a different dashboard and navigation set for each one. This approach is more detailed than a simple “one account, one role” design originally considered in the first version of our diagram and assumptions, but it matches the final implementation more closely and makes it easier to support the full workflow in one interface. At the same time, the project still limits users to the defined role set and adds checks to prevent duplicate registration for the same email-role combination.

4.5 Updated Entity-Relationship Diagram

The complete updated EER diagram for the RentCA system is presented in Figure 4.1. All entities, attributes, relationships, cardinality constraints, and specializations are shown using standard EER notation, including weak entities, composite attributes, and key attributes. The diagram reflects several refinements made between the initial design and the final implementation, driven either by a deeper understanding of the functional requirements or by issues uncovered while building the relational schema.

```

    erDiagram
        USER ||--o{ " " : ""
        USER ||--o{ PROSPECTIVE_RENTER : ""
        USER ||--o{ TENANT : ""
        USER ||--o{ APPOINTMENT : ""
        USER ||--o{ PROPERTY : ""
        USER ||--o{ RENTAL_LISTING : ""
        USER ||--o{ LEASE_RECORD : ""
        USER ||--o{ RENT_PAYMENT : ""
        USER ||--o{ MANAGEMENT_REQUEST : ""

        PROSPECTIVE_RENTER ||--1 TENANT : BOOKS
        PROSPECTIVE_RENTER ||--1 APPOINTMENT : ""
        PROSPECTIVE_RENTER ||--1 PROPERTY : MANAGES
        PROSPECTIVE_RENTER ||--1 RENTAL_LISTING : MANAGES_LISTING
        PROSPECTIVE_RENTER ||--1 LEASE_RECORD : SIGNS
        PROSPECTIVE_RENTER ||--1 MANAGEMENT_REQUEST : ""

        TENANT ||--1 APPOINTMENT : ""
        TENANT ||--1 PROPERTY : CONTAINS
        TENANT ||--1 LEASE_RECORD : BELONGS_TO
        TENANT ||--1 RENT_PAYMENT : PAYS
        TENANT ||--1 MANAGEMENT_REQUEST : FOR

        APPOINTMENT ||--1 PROPERTY : ""
        APPOINTMENT ||--1 MANAGEMENT_REQUEST : SUBMITS

        PROPERTY ||--1 RENTAL_LISTING : LISTED_AS
        PROPERTY ||--1 MANAGEMENT_REQUEST : FOR_PROPERTY
        PROPERTY ||--1 LEASE_RECORD : ""
        PROPERTY ||--1 RENT_PAYMENT : ""
        PROPERTY ||--1 MANAGEMENT_REQUEST : ""

        RENTAL_LISTING ||--1 LEASE_RECORD : ""
        RENTAL_LISTING ||--1 RENT_PAYMENT : ""
        RENTAL_LISTING ||--1 MANAGEMENT_REQUEST : HANDLES

        LEASE_RECORD ||--1 RENT_PAYMENT : ""
        LEASE_RECORD ||--1 MANAGEMENT_REQUEST : ""

        RENT_PAYMENT ||--1 MANAGEMENT_REQUEST : ""
  
```

The updated EER diagram is based on the following assumptions:

1. A User may hold one or more of three roles within the system: ProspectiveRenter, Tenant, and PropertyManager. Each role is represented by a separate USER record, and a single individual is identified across roles by a shared email value. For any given email, at most one record may exist per role; since there are three distinct roles, a single individual can hold up to three USER records in total.
2. The three USER roles are independent and are registered separately. A USER record's UserType is fixed at the time of creation and does not change during the user's lifecycle; to hold multiple roles, an individual registers a separate USER record for each, with all records sharing the same email.
3. Each Property is owned and managed by exactly one PropertyManager.
4. Each Property is uniquely identified by its physical address (Province, City, Street Name, Postal Code, Suite, Apartment). A composite uniqueness constraint on these attributes prevents the registration of two PROPERTY records describing the same physical location.
5. A PropertyManager manages one or more Properties.
6. A Property may have multiple RentalListings over time, but at most one active listing at any given moment.
7. Each RentalListing describes exactly one Property.
8. A ProspectiveRenter may book multiple Appointments.

9. Each Appointment is booked by at most one ProspectiveRenter and is associated with at most one RentalListing.
10. Each Appointment contains one or more TimeSlots, where each TimeSlot specifies a Day, Month, Year, and Hour. The TIME_SLOT entity restricts appointment booking to business hours (9–17).
11. The system prevents double-booking of the same TimeSlot across Appointments.
12. A LeaseRecord is associated with exactly one Property and exactly one Tenant, and covers a fixed time period (StartDate to EndDate).
13. A Tenant may hold multiple LeaseRecords over time, including concurrent leases on different Properties. A Property may have multiple LeaseRecords for different non-overlapping time periods.
14. Each RentPayment is associated with exactly one LeaseRecord.
15. A Tenant may submit multiple ManagementRequests.
16. Each ManagementRequest is associated with exactly one Property and is reviewed by the PropertyManager responsible for that Property.
17. Each ManagementRequest tracks both whether the tenant is authorised to submit it (Permission) and its workflow resolution (Status).

Changes since the initial EER diagram

1. Specialization constraint changed from disjoint to overlapping. The original presentation diagram modeled the USER specialization as disjoint (d), implying that a person could hold only one role at a time. The updated diagram uses overlapping (o) to reflect the requirement that one person may simultaneously hold multiple roles — for example, a property manager who also rents elsewhere as a tenant. This affects Assumption 1 (“A User may act as a ProspectiveRenter, Tenant, or PropertyManager”) and Assumption 2 (“A ProspectiveRenter becomes a Tenant upon signing a LeaseRecord”). Under the new design, a ProspectiveRenter does not transform into a Tenant when signing a lease; instead, the three roles are independent and are registered separately, with each role represented by its own USER record. A single individual is identified across roles by a shared Email value, and a composite uniqueness constraint (Email, UserType) ensures at most one record per role. At the application level, the user authenticates into one active role per session and may switch between their available roles, but all role records coexist in the database concurrently.

2. MaintenanceRequest renamed to ManagementRequest. The original entity MAINTENANCE_REQUEST was renamed to MANAGEMENT_REQUEST to better reflect the broader range of tenant-to-manager interactions it captures — maintenance issues, permission requests, and general inquiries — rather than maintenance work alone. The internal structure of the entity (including its Status and Permission fields) was already present in the original design and remains unchanged.

3. PROPERTY specialization collapsed into the IsRented attribute. The original {Rented, NotRented} specialization of PROPERTY was replaced with a single Boolean attribute IsRented directly on PROPERTY. Because the two subclasses had no distinguishing attributes or

relationships in the domain, the specialization was redundant. Moving the rental status to a Boolean attribute on the parent entity simplified the schema without losing any information.

4. PROPERTY extended with ManagerID attribute and address uniqueness constraint. A ManagerID attribute (foreign key to USER) was added explicitly to the PROPERTY entity, making the manager-property ownership relationship directly visible at the entity level. In parallel, a composite uniqueness constraint UNIQUE KEY uniq_property_address (Province, City, StName, PostCode, Suite, Appart) was added to the PROPERTY relation at the DDL level to prevent the registration of duplicate properties at the same physical location. The uniqueness constraint is enforced by the relational schema rather than shown as a visual annotation in the EER diagram; together with the ManagerID addition, it establishes Assumption 4.

5. Hour attribute added to TIME SLOT. TIME SLOT originally stored only Day, Month, and Year as part of its composite Date attribute. An Hour attribute (constrained to the range 0–23) was added so that booked slots are uniquely identifiable within a single day, and so that the system can prevent double-booking at hourly granularity. The frontend further restricts selectable hours to the range 9–17 to match standard business hours, while the database accepts any valid 24-hour value. This refinement strengthens Assumption 11 by giving it a concrete enforcement mechanism at the entity level.

6. LeaseRecord assumption refined. The original assumption stated that “A LeaseRecord is associated with exactly one Property and one Tenant”. The updated assumption was extended to make the non-overlap rule explicit: a Property may have multiple LeaseRecords scheduled for different non-overlapping time periods, while a Tenant may hold concurrent LeaseRecords across different Properties.

These changes together make the EER diagram fully consistent with the final relational schema and capture the design decisions made during implementation.

5. Implementation

Updated RM Diagram

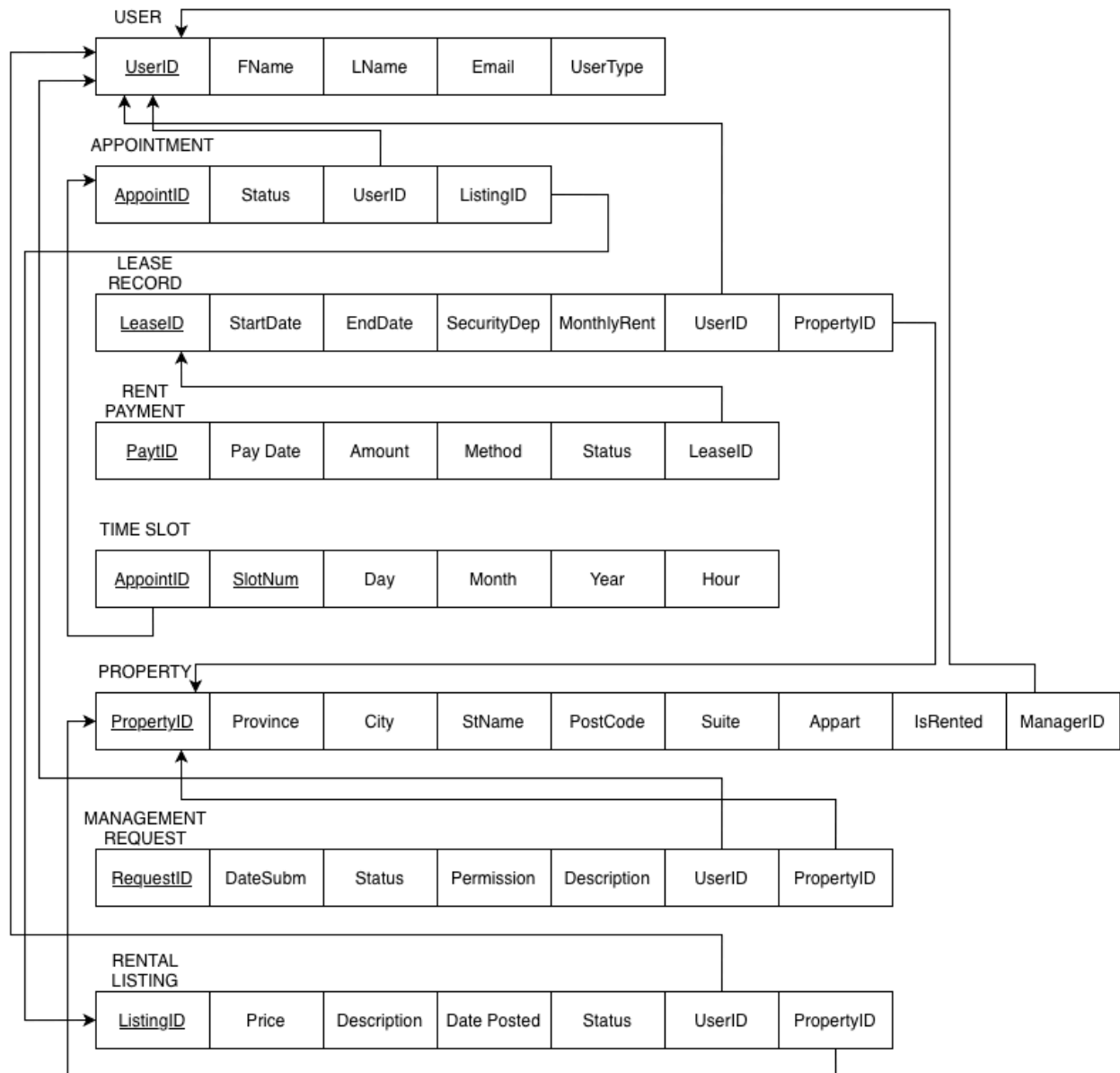


Figure 5.1: Updated RM Diagram

The relational schema was derived from the EERD by applying the standard ER-to-relation mapping algorithm. Strong entities (USER, APPOINTMENT, LEASE_RECORD, RENT_PAYMENT, PROPERTY, RENTAL_LISTING, MANAGEMENT_REQUEST) were converted into separate relations with their primary keys. The weak entity TIME_SLOT was given a composite primary key combining its owner's key (AppointID) with its partial key (SlotNum). Each binary 1:N relationship from the EERD was implemented by placing a foreign key on the "many" side. Both EER specializations were mapped using the single-relation

strategy: USER is implemented as one table with a UserType discriminator covering {ProspectiveRenter, Tenant, PropertyManager}, and the {NotRented, Rented} specialization of PROPERTY was collapsed into a single IsRented Boolean attribute within the existing PROPERTY relation, alongside its address fields and other attributes. The diagram in Figure 5.1 shows the resulting relations together with their primary keys, foreign keys, and the constraints added during implementation.

Assumptions:

1. The UserType attribute in USER is defined as an ENUM with possible values: ProspectiveRenter, Tenant, and PropertyManager. Each USER record has exactly one role, and the roles are mutually exclusive within a single record. However, the same physical person (identified by email) may correspond to multiple USER records — one per role they wish to perform — allowing a single individual to act as Property Manager, Tenant, and Prospective Renter through separate accounts that share an email but have distinct UserIDs. An email can be used only once per role: duplicate roles for the same email are not allowed, meaning a single person (one email) can hold at most three USER records where one for each of the three roles.
2. Property has an IsRented attribute of type Boolean, where TRUE indicates the property is rented and FALSE indicates it is not rented.
3. The Email attribute in USER is not globally unique — the same email may appear in multiple USER records, but only once per UserType. This means the combination (Email, UserType) is unique: an email can correspond to at most three USER records, one per role.
4. A PROPERTY may have multiple LEASE RECORD entries over time, but at most one LEASE RECORD may be active for a PROPERTY at any given time.
5. The system prevents double-booking of the same TIME SLOT for appointments.
6. The Hour attribute in TIME_SLOT restricts users to business hours (9-17) for usability.
7. Each RENT PAYMENT must reference an existing LEASE RECORD.
8. MANAGEMENT_REQUEST has a Status field that tracks the workflow state of the request (Pending/Approved/Rejected/Completed) and a Permission field that tracks whether the tenant is authorized to submit it (Granted/Denied/Pending). This lets the property manager separately control who may submit requests and how each request is resolved.

Changes from the original ER diagram:

1. PROPERTY.ManagerID — a foreign key to USER was added so that every property is explicitly linked to the Property Manager responsible for it.

2. PROPERTY address uniqueness — a composite UNIQUE constraint was added over (Province, City, StName, PostCode, Suite, Appart) to prevent the same physical address from being registered twice.
3. APPOINTMENT.ListingID — a foreign key to RENTAL_LISTING was added with ON DELETE SET NULL, so that appointment history is preserved even if a listing is later removed.
4. TIME_SLOT.Hour — an Hour attribute was added so that scheduling can be tracked at hour-level and double-booking can be prevented more reliably. The original design stored only the day.
5. USER (Email, UserType) UNIQUE — instead of treating Email as globally unique, a UNIQUE constraint was placed on the pair (Email, UserType), and Email was made NOT NULL. This allows one person to hold up to three role-specific accounts under the same email.
6. Multiple roles via separate USER records — the original design assumed a Prospective Renter would convert into a Tenant upon signing a lease. The implementation instead keeps each role as a separate USER record, which simplifies role-specific logic and supports the role-switching interface in the dashboard.

The final implementation of RentCA was built as a web application backed by a relational MySQL database. The system was implemented using Django and Django REST Framework on the backend and React on the frontend, with MySQL selected as the DBMS for persistent storage and enforcement of relationships between entities. This choice was appropriate for the project because the application depends heavily on structured relational data, including multiple linked entities, transaction-like updates, and constraints such as foreign keys, status values, and controlled user-role behavior. The final system was therefore implemented as a database-centered application rather than a frontend-only prototype.

5.1 Schema Discussion

The implemented schema is centered around eight main relations: USER, PROPERTY, RENTAL_LISTING, APPOINTMENT, TIME_SLOT, LEASE_RECORD, RENT_PAYMENT, and MANAGEMENT_REQUEST. These relations correspond directly to the core operations of the application and are linked using primary and foreign keys. The final relational model also includes assumptions such as unique user email addresses, one role per user row, at most one active lease per property at a time, and prevention of double-booking of the same appointment slot.

Several implementation decisions were significant during the transition from design to the final relationship schema. First, the system separates properties from rental listings. A property represents a physical unit, while a rental listing represents the property's advertised availability. This makes it possible for a property to remain in the database even when it is not currently listed, and it also allows listing status to change independently of the property record itself. Second, appointments are represented using two related relations: APPOINTMENT and TIME_SLOT. This separates the appointment record itself from the scheduled date information and keeps the booking design modular. The appointment row stores the user, listing, and status, while the time slot row stores the date and time associated with that appointment. In the later

stages of the project, the actual time (hour) was added alongside the date to support more specific appointment scheduling and stronger duplicate-booking prevention, reflecting the original goal of avoiding overlapping viewings. A third important implementation decision was the treatment of roles. In the final system, user roles are stored as separate user records rather than dynamically changing one user from Prospective Renter to Tenant. This differs from earlier assumptions in the planning stage, where a Prospective Renter automatically becomes a Tenant upon signing a lease. In the implemented version, a separate tenant-role must be added using the same account information, and lease creation links that tenant-role to a property. Although this is a compromise relative to the original assumption, it simplified role-specific logic and made the final role-switching interface in our web design easier to support.

The backend implementation follows this schema closely. Django models were used to define the database structure, and migrations were applied to create the corresponding tables in MySQL. The system also uses REST API endpoints to perform create, read, and update operations on these relations. In addition, the repository includes SQL schema and seed-data files so that the final dataset can be loaded consistently for demonstration and testing, and to help simulate a real-life rental management system.

5.2 DBMS Choice

MySQL was selected as the DBMS because the project depends on relational structure, persistent records, and clearly defined relationships between entities. The system stores users, properties, listings, appointments, leases, payments, and requests in separate but connected tables, making a relational DBMS a natural choice. MySQL also supports primary keys, foreign keys, enumerated values, and transactional updates, all of which are relevant to the implementation of this system. For example, appointment records must remain linked to users and listings, lease records must reference both a property and a tenant, and payment records must always reference an existing lease. These requirements are better supported by a relationship DBMS than by ad hoc file-based or in-memory storage.

Another advantage of MySQL in this project is that it integrates cleanly with Django. The backend was configured to use MySQL through Django's database settings, and the schema was created and maintained through Django migrations. This allowed the application logic and relational design to remain synchronized during implementation, while still supporting the use of SQL seed files for loading a complete demo dataset.

5.3 SQL Statements for Implemented Transactions

The RentCA system uses SQL operations through the Django backend and MySQL database to support core rental management workflows. The following examples represent key transactions implemented in the final system. These statements reflect the schema defined in the backend models and demonstrate how user actions are translated into relational database operations.

Create a User Account

- When a new user registers, a record is inserted into the USER table with a selected role.

</> SQL

```
INSERT INTO USER (FName, LName, Email, UserType)
```

```
VALUES ('Alex', 'Firth', 'alex@email.com', 'ProspectiveRenter');
```

- This creates a new account that can access role-specific functionality.

Add a Property

- Property Managers can create new rental properties.

</> SQL

```
INSERT INTO PROPERTY
```

```
(Province, City, StName, PostCode, Suite, Appart, IsRented, ManagerID)
```

```
VALUES
```

```
('Alberta', 'Calgary', 'Evanston View NW', 'T3P0H5', 'Unit 12', 'A', FALSE, 5);
```

- This stores a new rental unit managed by a property manager.

Submit a Management Request

- Tenants can submit maintenance or management issues.

</> SQL

```
INSERT INTO MANAGEMENT_REQUEST
```

```
(DateSubm, Status, Permission, Description, UserID, PropertyID)
```

```
VALUES
```

```
(CURDATE(), 'Pending', 'Pending',
```

```
'Kitchen sink is leaking.', 10, 12);
```

Update Request Status

- Managers can process submitted requests.

</> SQL

```
UPDATE MANAGEMENT_REQUEST
```

```
SET Status = 'Approved', Permission = 'Granted'
```

```
WHERE RequestID = 9;
```

- or

</> SQL

```
UPDATE MANAGEMENT_REQUEST
```

```
SET Status = 'Completed'
```

```
WHERE RequestID = 9;
```

Create a Rental Listing

- Managers can advertise an available property.

</> SQL

```
INSERT INTO RENTAL_LISTING
```

```
(Price, Description, DatePosted, Status, UserID, PropertyID)
```

```
VALUES
```

```
(1600.00, 'Modern apartment with parking and balcony.',
```

```
CURDATE(), 'Active', 5, 12);
```

- This makes the property visible to prospective renters.

Search / Filter Listings

- Prospective renters can browse active listings by city and price range.

</> SQL

```
SELECT RL.ListingID, RL.Price, P.City, P.StName
```

```
FROM RENTAL_LISTING RL
```

```
JOIN PROPERTY P ON RL.PropertyID = P.PropertyID
```

```
WHERE RL.Status = 'Active'
```

```
AND P.City = 'Calgary'
```

```
AND RL.Price BETWEEN 1200 AND 2000;
```

- This returns matching available rental listings.

Book an Appointment

- A prospective renter can request a viewing appointment.

</> SQL

```
INSERT INTO APPOINTMENT (Status, UserID, ListingID)
```

```
VALUES ('Pending', 8, 3);
```

Then the appointment time is stored.

```
INSERT INTO TIME_SLOT (AppointID, SlotNum, Day, Month, Year)
```

```
VALUES (15, 1, 24, 4, 2026);
```

- This creates a scheduled viewing request.

Prevent Duplicate Booking for the Same Slot

- Before inserting a new appointment, the system checks if the same property and time is already booked.

</> SQL

```
SELECT COUNT(*)
```

```
FROM APPOINTMENT A
```

```
JOIN TIME_SLOT T ON A.AppointID = T.AppointID
```

```
WHERE A.ListingID = 3
```

```
AND T.Day = 24
```

```
AND T.Month = 4
```

```
AND T.Year = 2026
```

```
AND A.Status IN ('Pending', 'Confirmed');
```

- If the result is greater than zero, the booking is rejected.

Update Appointment Status

- Managers can confirm, cancel, or complete appointments.

</> SQL

```
UPDATE APPOINTMENT
```

```
SET Status = 'Confirmed'
```

```
WHERE AppointID = 15;
```

```
UPDATE APPOINTMENT
```

SET Status = 'Completed'

WHERE AppointID = 15;

- This keeps renters informed of booking progress.

Create a Lease Record

- When a renter becomes a tenant, the manager creates a lease.

</> SQL

INSERT INTO LEASE_RECORD

(StartDate, EndDate, SecurityDep, MonthlyRent, UserID, PropertyID)

VALUES

('2026-05-01', '2027-04-30', 1000.00, 1600.00, 10, 12);

- This links a tenant to a rental property.

Submit a Rent Payment

- Tenants can submit rent payments.

</> SQL

INSERT INTO RENT_PAYMENT

(PayDate, Amount, Method, Status, LeaseID)

VALUES

(CURDATE(), 1600.00, 'Transfer', 'Pending', 7);

- Payments are reviewed by managers afterward.

Approve or Fail a Payment

- Managers update payment status after review.

</> SQL

UPDATE RENT_PAYMENT

SET Status = 'Completed'

WHERE PaytID = 22;

- or

```
UPDATE RENT_PAYMENT
```

```
SET Status = 'Failed'
```

```
WHERE PaytID = 22;
```

Update Request Status

- Managers can process submitted requests.

```
UPDATE MANAGEMENT_REQUEST
```

```
SET Status = 'Approved', Permission = 'Granted'
```

```
WHERE RequestID = 9;
```

- or

```
UPDATE MANAGEMENT_REQUEST
```

```
SET Status = 'Completed'
```

```
WHERE RequestID = 9;
```

Overall, these transactions demonstrate how the RentCA system uses SQL to manage user accounts, properties, listings, appointments, leases, payments, and maintenance requests while maintaining consistency through primary keys, foreign keys, and backend validation logic.

6. Visual Interface

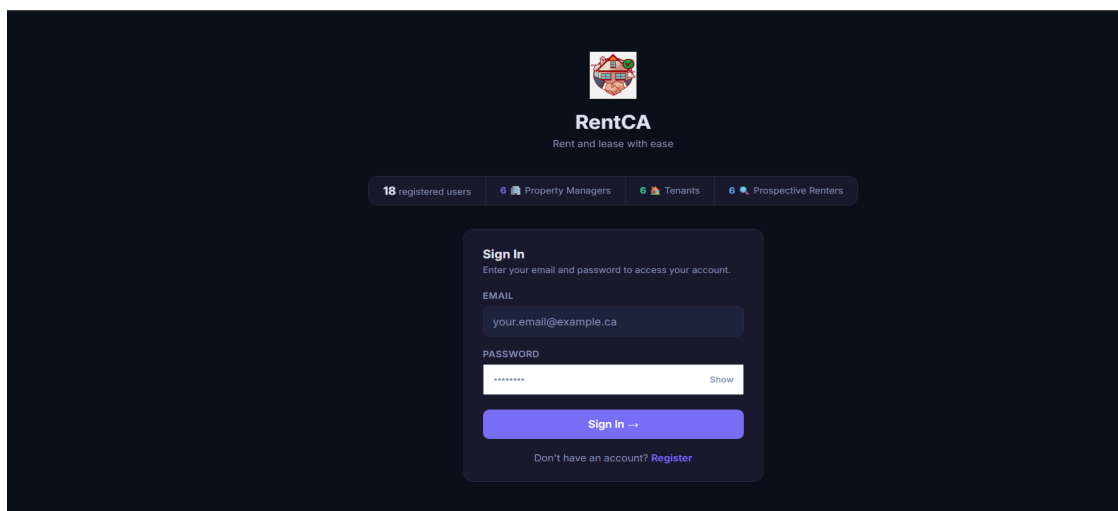


Figure 6.1: Login interface showing a simple and clean authentication form with email and password input fields.

18 registered users

6 Property Managers

6 Tenants

6 Prospective Renters

Create Account

Fill in your details to register.

FIRST NAME

Jane

LAST NAME

Smith

EMAIL

jane.smith@example.ca

PASSWORD

Show

ROLE

Prospective Renter

Prospective Renter

✓ Register & Sign In

Already have an account? Sign In

Figure 6.2: Account registration interface allowing users to enter personal details and select a role within the system

RentCA

Rent and lease with ease

MH

Maryna Hez

Property Manager

Maryna989898@gmail.com

ID: 1003

ACTIVE ROLE

Prospective Renter

Tenant

Property Manager active

NAVIGATION

Dashboard

Properties

Listings

Appointments

Leases

Payments

Requests

Sign Out

Welcome back, Maryna 🍷

PROPERTIES

20

total managed

ACTIVE LISTINGS

11

PENDING APPTS

4

ACTIVE LEASES

10

PENDING PAYMENTS

3

need approval

OPEN REQUESTS

5

awaiting action

Quick Actions

Use the sidebar to manage properties, listings, appointments, leases, payments, and maintenance requests.

Figure 6.3: Property Manager dashboard displaying key system metrics such as properties, listings, appointments, leases, payments, and requests

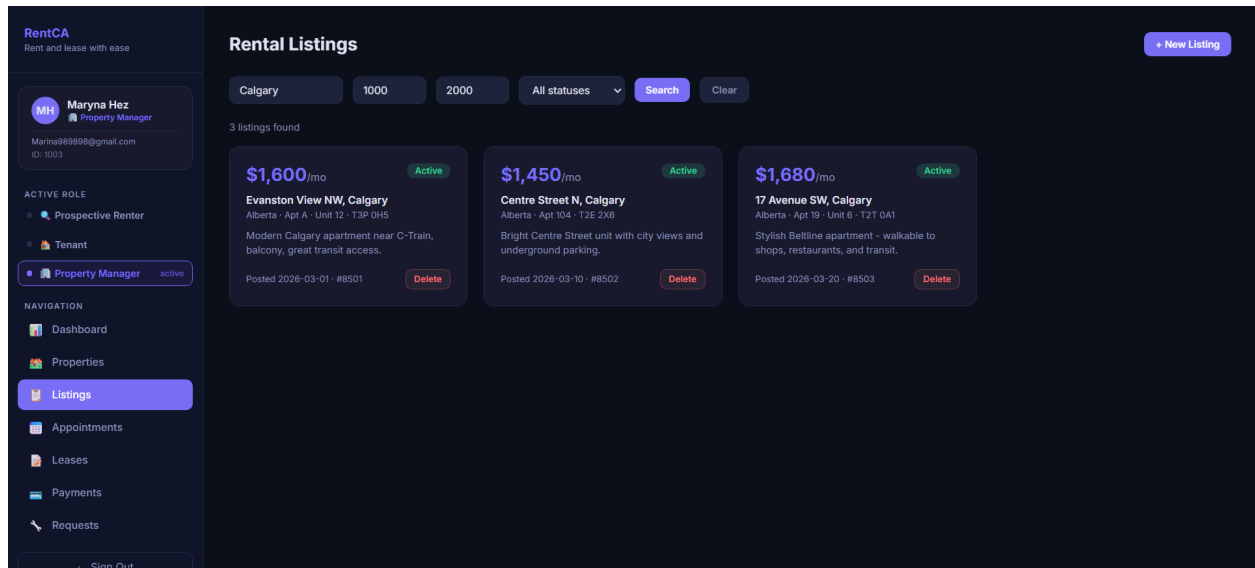


Figure 6.4: Rental listings interface with filtering options for city, price range, and listing status

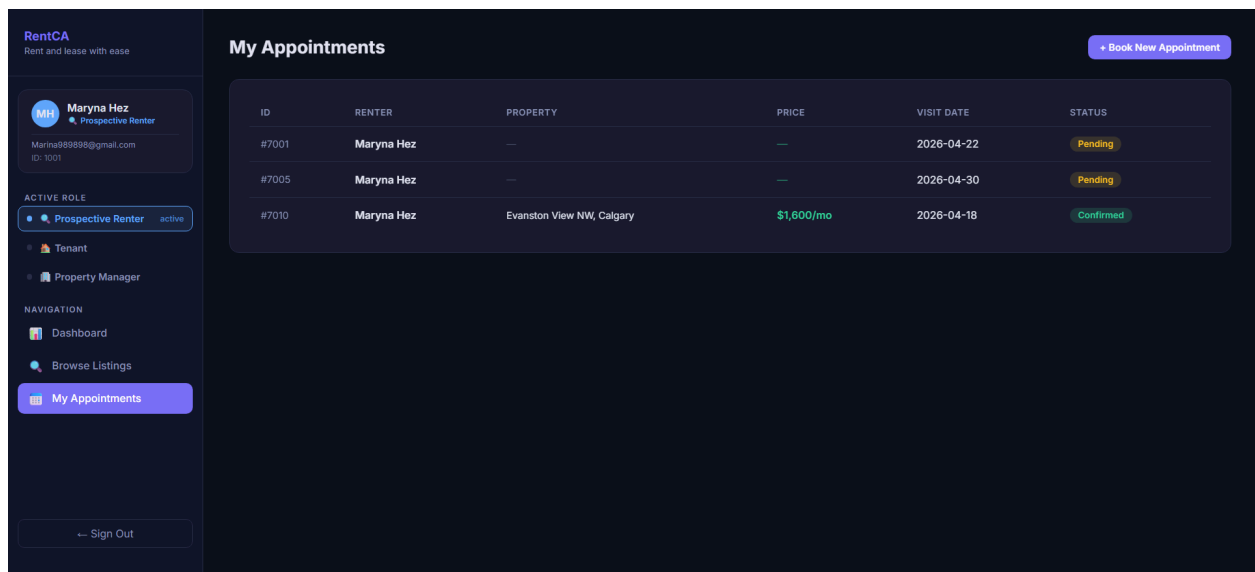


Figure 6.5: Prospective renter “My Appointments” interface showing scheduled viewings and their current status

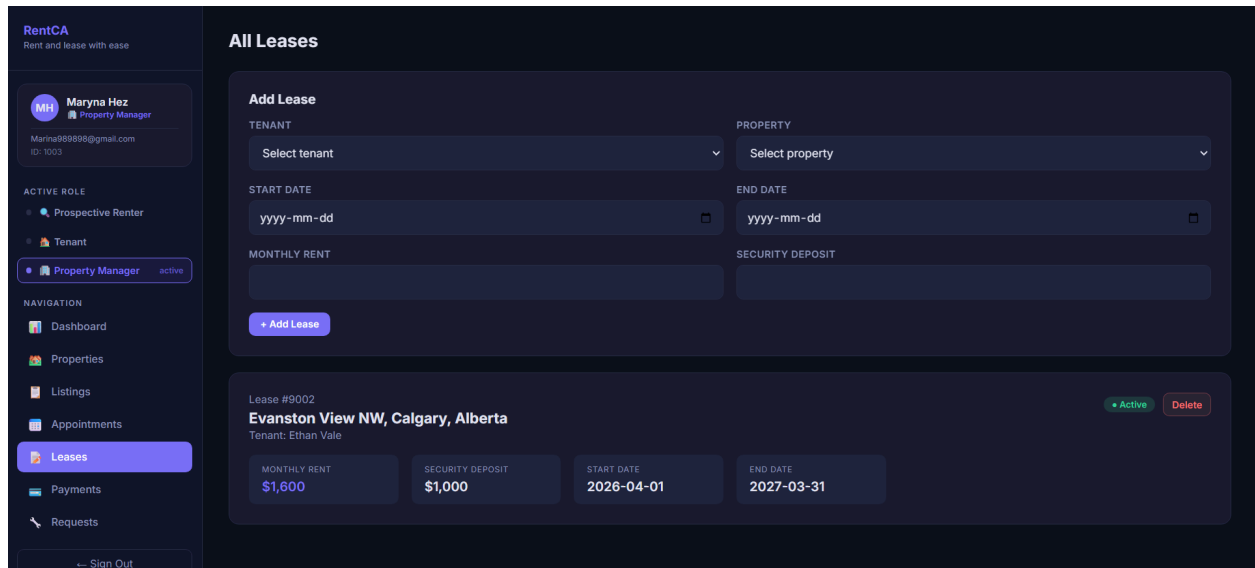


Figure 6.6: Lease management interface allowing property managers to create leases and view existing lease records

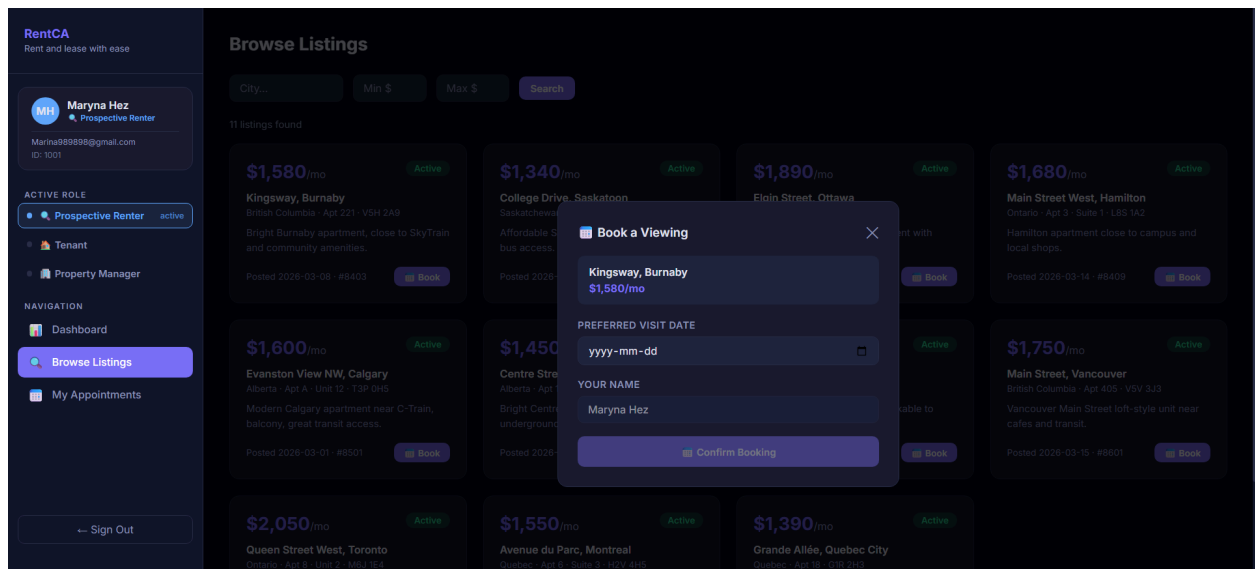


Figure 6.7: Appointment booking interface showing a model dialog with calendar-based date selection and pre-filled user information

6.1 User Interface and Usability

The RentCA interface is designed to be clear, consistent, and easy to navigate. The system uses a sidebar layout with a central content area, allowing users to move between features such as listings, appointments, leases, payments, and requests without losing context.

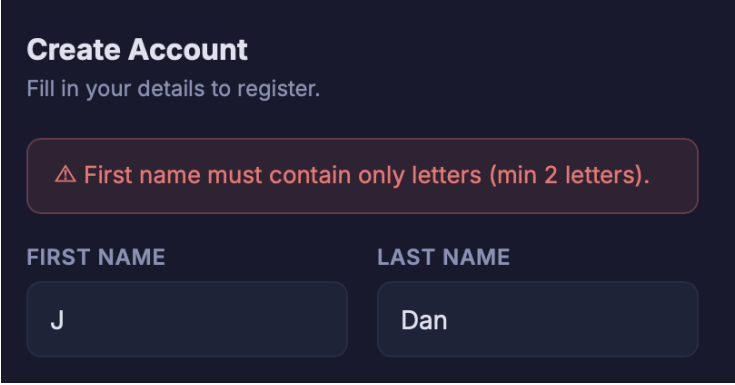
The interface adapts based on the user's active role, ensuring that only relevant functionality is displayed.

The login and registration pages are simple and require only essential information, making the system accessible to first-time users. Once logged in, users are presented with a role-specific dashboard that summarizes key system information, allowing them to quickly understand system activity without navigating multiple pages. Role switching within a single account further improves usability. The listing interface supports efficient browsing through filters such as city and price range, and users can book viewings directly from listings using a modal interface with a calendar-based date selector. The appointments interface provides clear feedback through status labels such as "Pending" and "Confirmed," allowing users to track their requests

6.2 Validation and Error Preventing

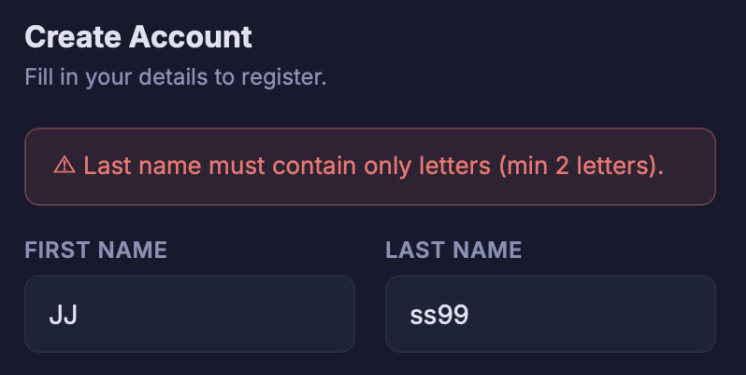
To improve reliability, several validation checks are enforced in the system. Each check provides immediate, clear feedback directly in the relevant form so users understand what went wrong and how to correct it:

- **Name validation:** First and last names must contain only letters and be at least two characters long. (Figure 6.8, 6.9)



The screenshot shows a dark-themed 'Create Account' form. At the top, the title 'Create Account' is in white, followed by the instruction 'Fill in your details to register.' in a lighter blue. Below this, a red-bordered box contains a warning icon (a triangle) and the message 'First name must contain only letters (min 2 letters)'. Underneath, there are two input fields. The 'FIRST NAME' field contains the letter 'J', and the 'LAST NAME' field contains 'Dan'.

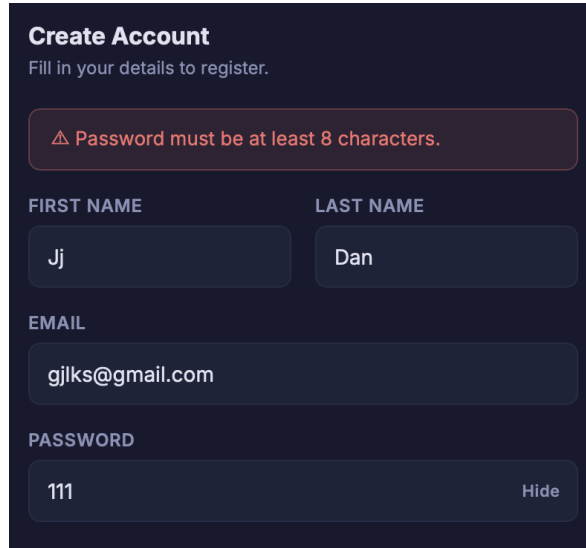
Figure 6.8: First name validation error message during account registration



The screenshot shows the same 'Create Account' form. The red-bordered warning box now displays the message 'Last name must contain only letters (min 2 letters)'. The 'FIRST NAME' field contains 'JJ', and the 'LAST NAME' field contains 'ss99'.

Figure 6.9: Last name validation error message during account registration

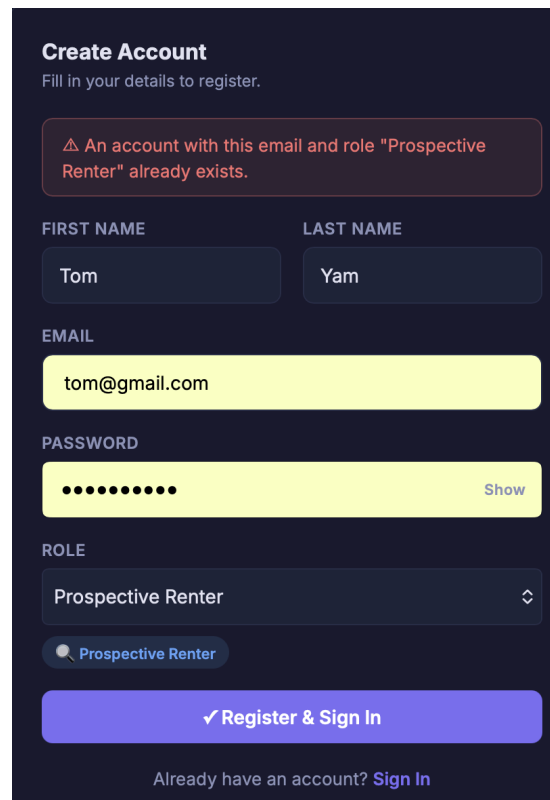
- **Password validation:** Passwords must be at least eight characters. (Figure 6.10)



The image shows a 'Create Account' form with a dark blue background. At the top, the title 'Create Account' is in white, followed by the instruction 'Fill in your details to register.' Below this is a red error message box with a triangle icon and the text 'Password must be at least 8 characters.' The form contains four input fields: 'FIRST NAME' with the value 'Jj', 'LAST NAME' with the value 'Dan', 'EMAIL' with the value 'gjlks@gmail.com', and 'PASSWORD' with the value '111'. The password field has a 'Hide' button next to it.

Figure 6.10: Password validation error message during account registration

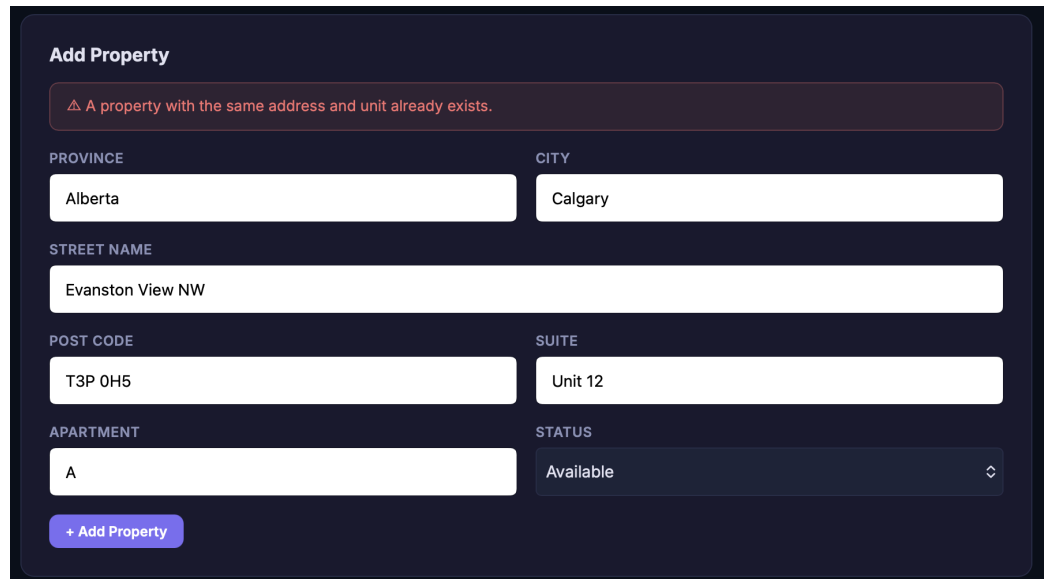
- **Duplicate account prevention:** If a user attempts to register with an email and role that already exists, the system displays an error immediately. (Figure 6.11)



The image shows a 'Create Account' form with a dark blue background. At the top, the title 'Create Account' is in white, followed by the instruction 'Fill in your details to register.' Below this is a red error message box with a triangle icon and the text 'An account with this email and role "Prospective Renter" already exists.' The form contains four input fields: 'FIRST NAME' with the value 'Tom', 'LAST NAME' with the value 'Yam', 'EMAIL' with the value 'tom@gmail.com', and 'PASSWORD' with a masked value '.....'. The password field has a 'Show' button next to it. Below the password field is a 'ROLE' dropdown menu with the value 'Prospective Renter' and a search icon. At the bottom of the form is a blue button with a checkmark icon and the text 'Register & Sign In'. Below the button is a link that says 'Already have an account? Sign In'.

Figure 6.11: Duplicate account error message during registration

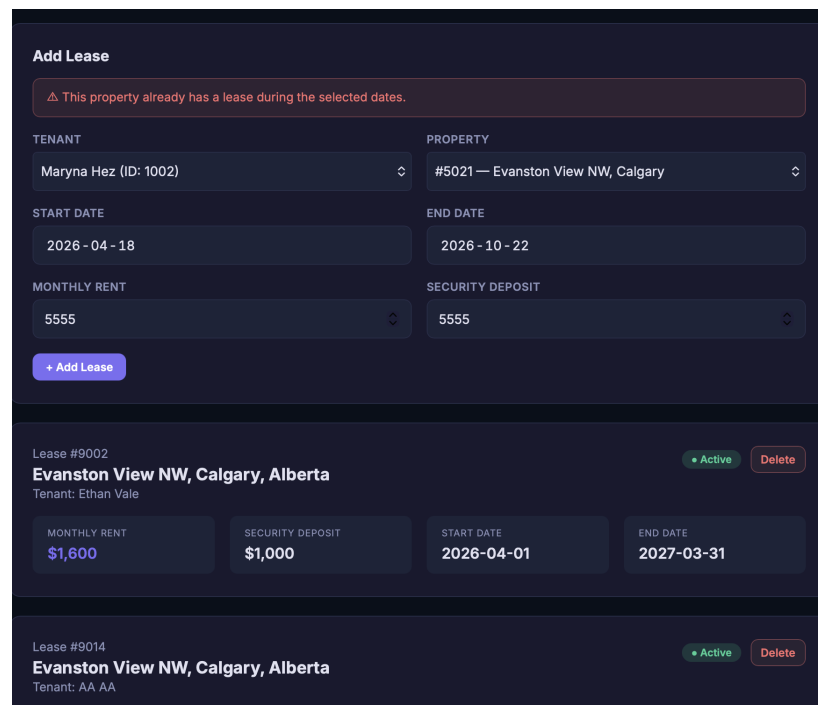
- **Duplicate property prevention:** If a property manager attempts to add a property with an identical address and unit, the form rejects the entry. (Figure 6.12)



The screenshot shows a dark-themed 'Add Property' form. At the top, a red error message bar states: '⚠ A property with the same address and unit already exists.' Below this, the form contains several input fields: 'PROVINCE' (Alberta), 'CITY' (Calgary), 'STREET NAME' (Evanston View NW), 'POST CODE' (T3P 0H5), 'SUITE' (Unit 12), 'APARTMENT' (A), and 'STATUS' (Available). A purple '+ Add Property' button is at the bottom.

Figure 6.12: Duplicate property error message in the Add Property form

- **Active lease conflict:** If a manager tries to assign a lease to a property that already has one active, the system blocks the action. (Figure 6.13)



The screenshot shows a dark-themed 'Add Lease' form. A red error message bar at the top states: '⚠ This property already has a lease during the selected dates.' The form includes fields for 'TENANT' (Maryna Hez (ID: 1002)), 'PROPERTY' (#5021 — Evanston View NW, Calgary), 'START DATE' (2026-04-18), 'END DATE' (2026-10-22), 'MONTHLY RENT' (5555), and 'SECURITY DEPOSIT' (5555). A purple '+ Add Lease' button is at the bottom.

Below the form, a list of existing leases for the property is shown:

Lease #	Tenant	Monthly Rent	Security Deposit	Start Date	End Date	Status
#9002	Ethan Vale	\$1,600	\$1,000	2026-04-01	2027-03-31	Active
#9014	AA AA					Active

Figure 6.13: Active lease conflict error in the Add Lease form

- **Double-booking prevention:** If a prospective renter selects a date and time already booked for the same property, the booking modal displays an error before submission. (Figure 6.14)

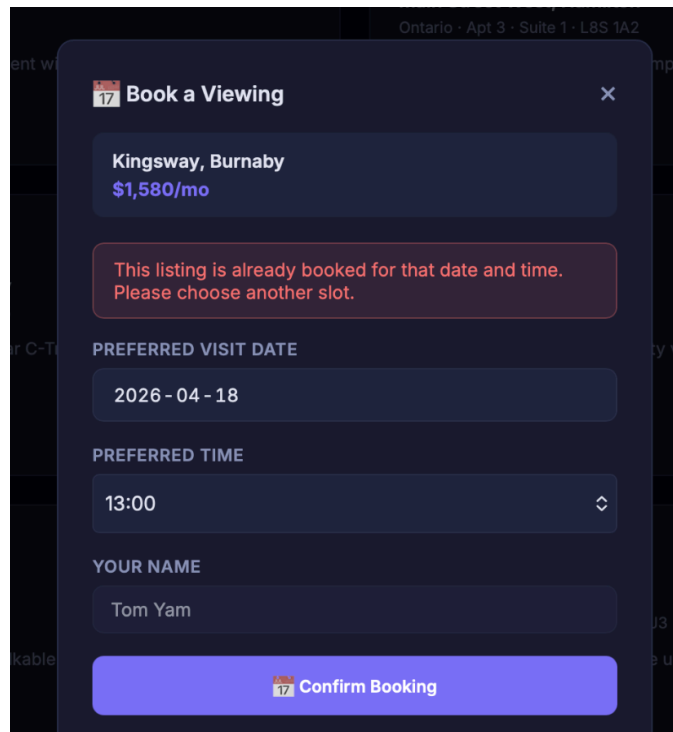


Figure 6.14: Double-booking error message in the Book a Viewing modal

Overall, the interface prioritizes usability by combining a consistent layout with simple interactions and targeted validation checks that help prevent errors.

7. User Guide

RentCA is a web-based rental property management system that supports three user roles: Property Manager, Tenant, and Prospective Renter. The system is accessed through a browser using the frontend interface, while data is stored in a MySQL database and managed through a Django REST backend.

7.1 Starting the System

To run the application locally:

1. Start the backend server from the backend directory using:

```
</> Bash: python manage.py runserver
```

2. Start the frontend server from the **frontend** directory using:

```
</> Bash: npm run dev
```

3. Open the frontend in a browser at:

```
http://localhost:3000
```

4. Backend runs at:

```
http://127.0.0.1:8000
```

7.2 Logging In

Users may log in using an existing account or create a new account through the registration page.

A demonstration account is included in the provided dataset:

- Email: Marina989898@gmail.com
- Password: 123456789A

This account contains Property Manager, Tenant, and Prospective Renter roles for testing system functionality.

7.3 Using the System by Role

Property Manager

After selecting the Property Manager role, users can:

- Create and manage properties
- Create rental listings
- View appointments
- Confirm, cancel, or complete bookings
- Create lease records
- Review rent payments
- Handle management requests

Tenant

After selecting the Tenant role, users can:

- View lease information
- Submit rent payments
- Review payment history

- Submit maintenance or management requests
- Track request status

Prospective Renter

After selecting the Prospective Renter role, users can:

- Browse available listings
- Filter listings by city or price
- Book viewing appointments
- View appointment status updates

7.4 Example Workflow

A typical workflow in the system is:

1. Property Manager creates a property
2. Property Manager creates a rental listing
3. Prospective Renter browses listings
4. Prospective Renter books a viewing appointment
5. Property Manager confirms the appointment
6. Property Manager creates a lease for the selected tenant
7. Tenant views lease details and submits rent payments

7.5 Validation and Error Prevention

The system includes validation rules to improve reliability:

1. Duplicate bookings for the same property and time are prevented
2. Duplicate accounts with the same email and role are blocked
3. Password length requirements are enforced
4. Duplicate property entries are restricted

7.6 Troubleshooting

If the interface appears empty or login fails:

1. Ensure the MySQL dataset has been loaded
2. Confirm backend and frontend servers are running
3. Verify database credentials in Django settings

8. Conclusion

This project presented the design and implementation of RentCA, a rental property management system developed to address common inefficiencies in organizing rental operations. By integrating listings, appointment scheduling, leases, payments, and management requests into a single system, the application replaces fragmented and manual processes with a centralized, structured solution.

The system demonstrates how a relational database can support real-world workflows through clearly defined entities and relationships, while the web-based interface provides an accessible way for users to interact with the data. Key features such as appointment scheduling, role-based interaction, and validation checks contribute to improving consistency and reducing errors in the rental process. Although the system successfully meets its primary goals, there are areas for future improvements, including more advanced authentication (such as Multi-Factor Authentication for greater security), tighter integration between user roles, and additional validation at the database level. Overall, the project highlights the effectiveness of combining database design with application development to solve a practical, real-world problem.